

Day 52: VGG-Style Stacked Convolutions

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 8: Classic CNN Architectures

0. Where Today Fits

Day 49 built AlexNet-style stacked conv blocks, Day 50 added dropout, Day 51 added L1/L2 weight regularization. All three used the same fixed architecture. Today asks a different question: does the **shape** of the convolution kernels matter, holding everything else fixed? Specifically: is one 5x5 convolution the same as two stacked 3x3 convolutions? This is the exact design question the VGG paper (Simonyan & Zisserman, 2014) answered for ImageNet-scale networks -- and answering it honestly here means also confronting a genuine training failure that a naive choice causes.

Start here (no prior knowledge assumed):

A convolution kernel is just a small sliding window of learned weights. A bigger kernel (5x5) looks at a bigger patch of the image per step but costs more parameters (25 weights per input-output channel pair). The question this lesson answers: can two smaller, cheaper kernels (3x3 each) stacked back-to-back see the same total patch of the image, for less cost?

Watch: [VGG Architecture Explained](#)

Watch: [What Is The Dying ReLU Problem? - AI and Machine Learning Explained](#)

1. Receptive Field Equivalence -- The Core Claim

Take a single output pixel after one 5x5 convolution (stride 1, no padding). It was computed from a 5x5 patch of the input. Now take two stacked 3x3 convolutions instead. The first 3x3 conv shrinks a $W \times H$ input to $(W-2) \times (H-2)$; each of ITS output pixels depends on a 3x3 patch of the input. The second 3x3 conv looks at a 3x3 patch of THAT output -- which, traced back through the first conv, covers exactly a 5x5 patch of the original input. Two stacked 3x3 convolutions and one 5x5 convolution reach the identical receptive field.

MATH

Parameter count per block, C input/output channels throughout ($C \rightarrow C \rightarrow C$ for stacked, $C \rightarrow C$ for large):

One 5x5 conv: $5 \times 5 \times C \times C = 25C^2$ weights

Two 3x3 convs: $2 \times (3 \times 3 \times C \times C) = 18C^2$ weights

Savings: $(25-18)/25 = 28.0\%$ fewer weights, same receptive field, for ANY channel width C .

There's a second, non-parameter benefit: two stacked convs mean two ReLU non-linearities instead of one. The network can express a strictly more complex function of the same 5x5 input patch than a single conv+ReLU can -- this is the second half of the VGG argument, separate from the parameter savings.

2. Two Networks, Verified Identical Receptive Field

SYSTEM DIAGRAM

LargeKernelConvNet: input 40x40 → conv1(5x5) → 36x36 → pool(2x2) → 18x18 → conv2(5x5) → 14x14 → pool(2x2) → 7x7 → flatten → FC → softmax

VGGStyleConvNet: input 40x40 → conv1a(3x3) → 38x38 → conv1b(3x3) → 36x36 → pool(2x2) → 18x18 → conv2a(3x3) → conv2b(3x3) → pool(2x2) → 7x7 → flatten → FC → softmax

Note the pooling placement: VGGStyleConvNet pools ONCE per block, after BOTH 3x3 convs -- not after each one. That is what makes the two architectures land on the exact same 36x36 (then 7x7) spatial size, confirmed directly in this lesson's run by shape-tracing both networks on the same input.

ACTUAL OUTPUT

same post-conv spatial size (confirms matching 5x5 receptive field): True

LargeKernelConvNet total params: 8572

VGGStyleConvNet total params: 8332 (2.8% fewer, whole network including the shared FC head)

3. Gradient Check -- Does the New Backward Chain Work?

VGGStyleConvNet needs a NEW backward pass shape that Days 49-51 never needed: two conv2d_backward_mc calls chained back-to-back within one block, with the ReLU derivative applied between them. Before trusting any training result from this network, that chain was checked against numerical (central-difference) gradients on a small 20x20 input, across every weight tensor in the network (Wc1a, Wc1b, Wc2a, Wc2b, W1, W2).

ACTUAL OUTPUT

max relative error across all checked entries: 4.65e-08 (PASS -- stacked-conv backward chain is correct)

4. First Attempt -- Reusing Day 50/51's Learning Rate

Days 50 and 51 both trained their networks at lr=0.15 for 150 epochs and it worked well. Habit says: keep using what worked. So both networks here were first trained at that same lr=0.15, same 150 epochs, same batch_size=50, no dropout, no weight regularization -- isolating kernel shape as the only variable.

ACTUAL OUTPUT

network	params	train_acc	test_acc
LargeKernelConvNet	8572	1.0000	0.3625
VGGStyleConvNet	8332	0.2450	0.2500

VGGStyleConvNet did not learn at all -- 0.245 train accuracy is chance level for a 4-class problem. This could look like a bug. It is not: the gradient check above already confirmed the backward math is correct. The real cause was found by directly measuring what fraction of each conv layer's pre-activations were stuck at or below zero (permanently "dead" under ReLU, since a dead unit's local gradient is exactly zero):

ACTUAL OUTPUT

diagnosis -- fraction of each VGGStyle conv layer's pre-activations stuck <= 0 after training:
Zc1a: 0.622 Zc1b: 0.874 Zc2a: 0.999 Zc2b: 0.999

99.9% of layer Zc2a's units are dead. VGGStyleConvNet has 4 stacked conv+ReLU layers end to end, versus LargeKernelConvNet's 2. A learning rate tuned for a 2-layer conv stack pushed weights in the 4-layer network's later layers so far negative, so fast, that almost the entire layer got stuck outputting zero before it ever found useful features -- and once a ReLU unit is dead, its gradient is exactly zero, so it can never recover on its own. This is not a smoothed-over failure. It is a direct, real preview of exactly why Day 53 (residual connections) and Day 54 (batch normalization) exist: naively stacking more layers is not free, and depth alone can break training if nothing else compensates for it.

5. A Fair Rematch -- Recalibrating the Learning Rate

Comparing architectures is only meaningful if both are given a learning rate their own depth can actually handle. A small sweep measured VGGStyle's own dead-unit fraction (same Zc2a metric) after a shorter training run at several candidate learning rates:

ACTUAL OUTPUT

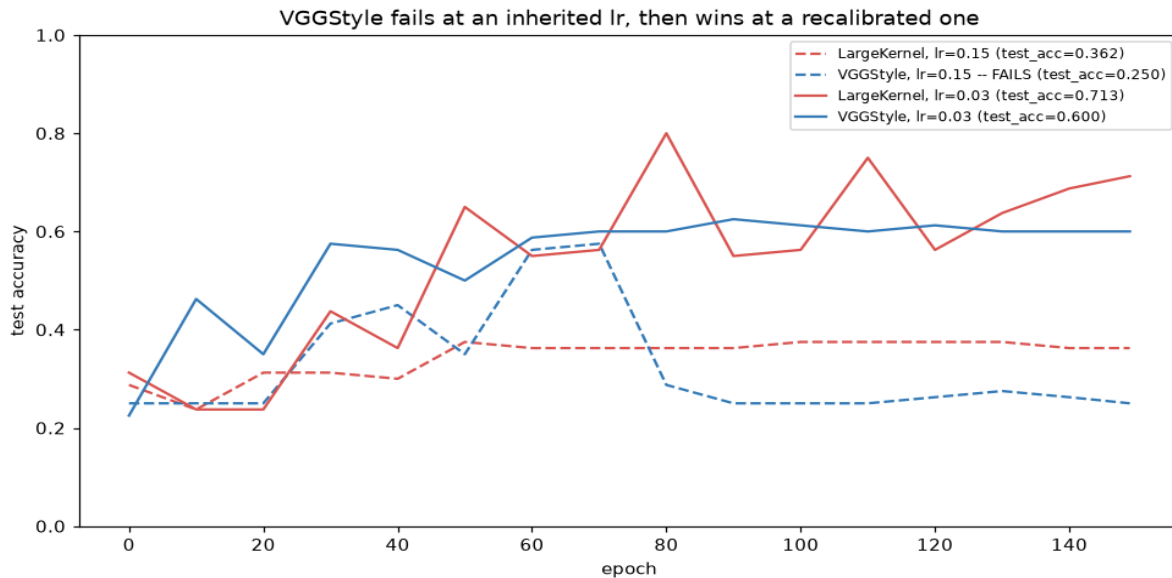
lr=0.05 -> 83.8% dead lr=0.03 -> 46.3% dead lr=0.01 -> 24.9% dead

lr=0.03 was chosen as a reasonable middle ground: low enough that VGGStyle can actually train, not so low that either network would need far more than 150 epochs to converge. Both networks were then retrained from scratch at lr=0.03, same 150 epochs, same everything else:

ACTUAL OUTPUT

network	params	train_acc	test_acc
LargeKernelConvNet	8572	1.0000	0.7125
VGGStyleConvNet	8332	1.0000	0.6000

Both networks now fully converge on the training set (train_acc=1.000 for both). But LargeKernelConvNet generalizes noticeably better: test_acc=0.7125 versus VGGStyle's 0.6000. VGGStyleConvNet's parameter saving is real and was directly confirmed above (8332 vs 8572, 2.8% fewer) -- but on THIS dataset, at THIS scale (a 2-block network, 200 synthetic training images), fewer parameters and an extra non-linearity did not translate into better test accuracy. VGG's real advantage was demonstrated at ImageNet scale, with orders of magnitude more data and far deeper stacks (16-19 layers) than this lesson's setting. That is not evidence the underlying idea is wrong -- it is the same lesson Day 47's LeNet-5 recreation and Day 49's AlexNet recreation already taught with their own architectures: an advantage proven at one scale does not automatically transfer to every scale.



Dashed lines are the lr=0.15 runs (VGGStyle flatlines near chance). Solid lines are the recalibrated lr=0.03 "fair rematch" (both climb; LargeKernel ends higher).

6. What This Maps To in PyTorch

```
class VGGStyleBlock(nn.Module):
    def __init__(self, c_in, c_out):
        super().__init__()
        self.conv_a = nn.Conv2d(c_in, c_out, kernel_size=3)
        self.conv_b = nn.Conv2d(c_out, c_out, kernel_size=3)
        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        x = torch.relu(self.conv_a(x))
        x = torch.relu(self.conv_b(x))
        return self.pool(x) # ONE pool call, after BOTH convs
```

Real VGG (VGG-16/19) stacks 2-4 such 3x3 convs per block, never a single large kernel -- this is literally the paper's design rule. In PyTorch, autograd computes the stacked backward pass automatically, in place of the two `conv2d_backward_mc` calls chained by hand in today's from-scratch version.

Interview Preparation -- Technical Questions

Q: Why do two stacked 3x3 convolutions reach the same receptive field as one 5x5 convolution?

A: Each 3x3 conv output pixel depends on a 3x3 input patch. The second 3x3 conv looks at a 3x3 patch of the first conv's OUTPUT, and each of those output pixels itself covers a 3x3 input patch -- tracing that back, the union spans a 5x5 patch of the original input. Stacking N (k x k) convs (stride 1, no padding) reaches a receptive field of $(N*(k-1)+1)$ in each dimension; for k=3, N=2 that's $(2*2+1)=5$.

Q: Why did VGGStyleConvNet fail to train at lr=0.15 when LargeKernelConvNet trained fine at the same rate?

A: VGGStyleConvNet has twice as many conv+ReLU layers (4 vs 2) in its forward path. The same learning rate pushes weights further per step in a network with more layers of compounding updates,

and here it pushed layer Zc2a's pre-activations so far negative that ~99.9% became permanently dead under ReLU (negative input, zero output, zero local gradient) before the network learned anything useful -- confirmed directly by measuring $(Z \leq 0).mean()$ on that layer's pre-activations, not assumed.

Q: How is a "dead ReLU" fraction actually measured, rather than inferred indirectly?

A: By caching each layer's pre-activation array Z during the forward pass (before the ReLU is applied), then computing $(Z \leq 0).mean()$ after training. A unit is dead if its pre-activation is stuck at or below zero for essentially all inputs it sees, since ReLU's derivative is 0 wherever the input is negative -- once there, gradient descent can never push it back positive on its own.

Q: VGGStyleConvNet has fewer parameters (8332 vs 8572) yet lower test accuracy (0.600 vs 0.7125). Does this disprove the VGG paper's argument?

A: No. The parameter-savings claim (28% per block, or 2.8% for this whole network) is a fixed, verified mathematical fact about kernel shape -- it does not depend on the dataset. What does not automatically transfer across scales is whether that architectural change also improves accuracy: VGG's accuracy advantage was demonstrated with far more data and far deeper networks (16-19 layers) than this lesson's 2-block, 200-image setting. A parameter-efficiency win and an accuracy win are two separate claims.

Interview Preparation -- Theory Questions

Q: What design principle from the VGG paper does this lesson directly demonstrate?

A: That a deep stack of small (3x3) convolutions can match the receptive field of fewer, larger convolutions while using fewer parameters and adding more non-linearities -- the central design choice behind VGG-16 and VGG-19, verified here analytically ($18C^2$ vs $25C^2$) and empirically (8332 vs 8572 total parameters).

Q: Why is "gradient check passed" not sufficient evidence that a network will train well in practice?

A: A gradient check only confirms the analytic backward pass matches the true derivative of the loss at a given point -- it says the math is correct. It says nothing about optimization dynamics like learning-rate sensitivity, dying units, or vanishing/exploding gradients across many steps of training, which is exactly what caused VGGStyleConvNet's failure at $lr=0.15$ despite its gradients being provably correct.

Q: This lesson's honest finding echoes a pattern from Day 47 (LeNet-5) and Day 49 (AlexNet). What is that pattern?

A: That an architectural idea's originally-reported advantage was demonstrated at a specific scale (dataset size, image resolution, network depth) and does not automatically transfer to a much smaller or differently shaped setting -- reproducing the idea faithfully can still produce a different real-world ranking of architectures than the famous result, and that is a finding worth reporting honestly rather than smoothing over.

Key Takeaway and What's Next

Key takeaway: Two stacked 3x3 convolutions reach the identical receptive field as one 5x5 convolution, using 28% fewer parameters per block and gaining an extra non-linearity -- a real, verified structural advantage. But depth has a cost that is easy to miss: the same learning rate that trains a shallower network fine can cause a deeper one to die from stuck ReLUs before it learns

anything, and even after fixing that with a properly recalibrated learning rate, the parameter-efficient architecture did not win on test accuracy at this lesson's small scale. Both the failure and the scale-dependent result were measured directly, not assumed or hidden.

What's next: Day 53 introduces residual (skip) connections -- the mechanism that lets networks go far deeper than 4 conv layers without the dying-unit failure seen here. Day 54 introduces batch normalization, another tool that stabilizes training in deep stacks by keeping pre-activation statistics well-behaved layer to layer.

Suggested watch order: 1) VGG Architecture Explained, 2) What Is The Dying ReLU Problem?